

Lecture 13: Dimensionality reduction

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will

1. Understand the **curse of dimensionality** and why we need reduction
2. Explain how **PCA** finds directions of maximum variance
3. Understand **t-SNE** and how it preserves local structure
4. Apply **UMAP** for fast, scalable dimensionality reduction
5. Compare different techniques and follow visualization best practices

The goal

Reduce from d dimensions (e.g., 300) to k dimensions (e.g., 2 or 3) while preserving as much meaningful structure as possible.

The curse of dimensionality

The problem

High-dimensional spaces behave in counter-intuitive ways that make visualization and computation difficult.

Embedding dimensions

- **Word2Vec / GloVe / FastText:** 300 dimensions
- **BERT:** 768 dimensions
- **GPT-3:** 12,288 dimensions!

Distance concentration

In high dimensions, all points tend to cluster around the same distance from the origin!

```
1 import numpy as np
2 # Sample 1000 random points in different dims
3 for dim in [2, 10, 100, 300]:
4     points = np.random.randn(1000, dim)
5     dists = np.linalg.norm(points, axis=1)
6     print(f"Dim={dim}: mean={dists.mean():.2f}, std={dists.std():.2f}")
7
```

Why dimensionality reduction?

Visualization

- 2D/3D plots to explore semantic structure
- Identify clusters and discover patterns
- Quality assurance for embeddings

Computation & ML

- **Faster algorithms:** Less memory and compute required
- **Reduce overfitting:** Feature selection and noise removal
- **Scalability:** Enable real-time systems

Interpretation

Understand relationships between words/documents and communicate results effectively.

Principal Component Analysis (PCA)

The classic linear method

Finds directions of maximum variance and projects data onto these orthogonal "principal components."

Properties

- **Linear transformation:** Preserves global structure
- **Deterministic:** Same result every time
- **Fast:** Efficient computation via SVD
- **Interpretable:** PCs are ranked by variance explained

Intuition

- **PC1:** Captures the most variation in the data
- **PC2:** Captures the remaining variation (orthogonal to PC1)

PCA: The algorithm

Worked example (3D → 2D)

Word	x1	x2	x3
king	2.0	1.5	0.1
queen	1.8	1.6	0.2
man	1.0	0.5	0.1
woman	0.9	0.6	0.2

Step 1: Center data (subtract mean)

Step 2: Compute covariance matrix

Step 3: Find eigenvectors/eigenvalues

Variance retained

- $\lambda_1 = 0.85$ (85% variance)
- $\lambda_2 = 0.13$ (13% variance)
- $\lambda_3 = 0.02$ (2% variance)

Keep top 2 PCs → 98% variance retained! Gender remains separable and royalty still clusters.

PCA in Python

Using scikit-learn

```
1  from sklearn.decomposition import PCA
2  from sklearn.preprocessing import StandardScaler
3
4  # Standardize the data (recommended)
5  scaler = StandardScaler()
6  embeddings_scaled = scaler.fit_transform(embeddings)
7
8  # Apply PCA
9  pca = PCA(n_components=2)
10 embeddings_2d = pca.fit_transform(embeddings_scaled)
11
12 # Check variance explained
13 print(f"Total variance explained:
      {sum(pca.explained_variance_ratio_):.2%}")
```

Visualization

PCA limitations

Assumes linearity

- Only captures linear relationships
- Word embeddings often have non-linear structure (manifolds)
- May miss important semantic patterns

Other issues

- **Variance $\neq$ Importance:** Noisy dimensions might have high variance
- **Global focus:** May lose local structure (clusters become blurred)
- **Interpretability:** PCs are linear combinations, hard to map to specific meanings

When to use PCA

Quick first exploration, preprocessing for other methods (like t-SNE/UMAP), or when speed is critical.

t-SNE: t-Distributed Stochastic Neighbor Embedding

Further reading

van der Maaten & Hinton (2008, *JMLR*) Visualizing Data using t-SNE.

Non-linear visualization

Models similarity as probability: Gaussian in high-D, t-distribution in low-D. Minimizes KL divergence between distributions.

Advantages

- Beautiful visualizations that reveal clusters
- Preserves local structure (neighbors stay neighbors)
- Widely used for high-dimensional data exploration

t-SNE: Key concepts

The crowding problem

In 2D, there isn't enough "room" to fit all the neighbors a point had in high-D. The **t-distribution** has heavier tails, allowing moderately-distant points to spread further apart.

Perplexity

Balances attention between local and global aspects of the data.

- **Low (5):** Tight, fragmented clusters (fine structure)
- **Medium (30):** Balanced view (standard choice)
- **High (100):** Merged clusters (global view)

Rule of thumb: Perplexity $\approx \sqrt{N}$

t-SNE in Python

Using scikit-learn

```
1 from sklearn.manifold import TSNE
2
3 tsne = TSNE(
4     n_components=2,
5     perplexity=30,
6     n_iter=2000,
7     random_state=42
8 )
9
10 embeddings_2d = tsne.fit_transform(embeddings)
```

Pro tip

t-SNE is computationally expensive. For large datasets, use PCA to reduce to 50 dimensions first, then run t-SNE.

t-SNE limitations and caveats

Performance & Determinism

- **Slow:** $O(N^2)$ complexity (10k points can take 15+ minutes)
- **Non-deterministic:** Different random seeds produce different layouts
- **No out-of-sample:** Cannot transform new points without refitting everything

Interpretation pitfalls

- **Cluster sizes are arbitrary:** A "bigger" cluster doesn't mean more variance
- **Distances between clusters are meaningless:** Global structure is not preserved
- **Axes have no meaning:** Don't try to label the x or y axis!

UMAP: Uniform Manifold Approximation & Projection

Further reading

McInnes et al. (2018, [arXiv](#)) UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction.

The modern standard (2018)

Theoretically grounded in topology. Faster than t-SNE and preserves both local AND global structure.

Key advantages

- **Fast:** $O(N \log N)$ complexity
- **Scalable:** Handles millions of points
- **Out-of-sample:** Can transform new points
- **Global structure:** Better at keeping related clusters near each other

UMAP in Python

Using umap-learn

```
1 import umap
2
3 reducer = umap.UMAP(
4     n_components=2,
5     n_neighbors=15, # Local vs. global balance
6     min_dist=0.1,   # Cluster tightness
7     metric='cosine', # Best for word embeddings
8     random_state=42
9 )
10
11 embeddings_2d = reducer.fit_transform(embeddings)
12
13 # Transform new points!
14 new_2d = reducer.transform(new_embeddings)
```

Hyperparameters

PCA vs. t-SNE vs. UMAP

Feature	PCA	t-SNE	UMAP
Speed	Very Fast	Slow	Fast
Scalability	Excellent	Poor (<10k)	Excellent
Global structure	Yes	No	Yes
Local structure	Partial	Yes	Yes
Embedding			

Visualization best practices

Preparation

- **Standardize:** Use `StandardScaler` for PCA
- **Sample:** For very large datasets, sample or use PCA first
- **Iterate:** Try multiple hyperparameters (perplexity, n_neighbors)

Presentation

- **Color intelligently:** By semantic category, cluster, or POS tag
- **Interactive:** Use Plotly or Bokeh for hover tooltips and zooming
- **Annotate selectively:** Label representative words, not every point

Interactive Plotly

```
1 import plotly.express as px
2 fig = px.scatter(df, x='x', y='y', color='category', hover_data=
  ['word'])
3 fig.show()
```

Applications in NLP

Embedding quality check

Quick sanity check: Do "dog", "cat", "fish" cluster together? If it's a random scatter, your embeddings might be poor.

Finding polysemy

"Apple" might appear in two clusters: one near "fruit" and another near "tech" (Google, Microsoft).

Model layer comparison

Visualize different BERT layers to see how syntax (lower layers) evolves into semantics (middle layers).

Case study: Visualizing BERT layers

Further reading

Jawahar et al. (2019, ACL) What Does BERT Learn about the Structure of Language?

Findings

- **Layers 0-2:** Syntactic (POS tags cluster)
- **Layers 3-8:** Semantic (meaning clusters)
- **Layers 9-12:** Task-specific adaptation

Insight

Confirms a linguistic hierarchy where the model builds complex meaning from basic structural components.

Discussion: What do 2D views tell us?

Information loss

Compressing 300D to 2D involves >99% information loss. Projections are "shadows" of the true high-D structure.

What we CAN conclude

- Rough semantic groupings and relative proximities
- Existence of clusters and outliers
- Qualitative patterns in the data

What we CANNOT conclude

- Exact distances or true high-D geometry
- Quantitative relationships between distant clusters
- Absolute cluster sizes (especially in t-SNE)

Summary

What we learned

1. **Curse of Dimensionality:** High-D space is counter-intuitive; reduction is essential for visualization.
2. **PCA:** Linear, fast, global focus. Best for noise removal and preprocessing.
3. **t-SNE:** Non-linear, local focus. Great for clusters but slow and non-deterministic.
4. **UMAP:** Fast, scalable, preserves local and global structure. The modern default.
5. **Workflow:** PCA (50D) → UMAP (2D) is the gold standard for NLP visualization.

Coming up

Next: Cognitive models of semantic representation — how do humans represent meaning?

Further reading

Foundational papers

van der Maaten & Hinton (2008, *JMLR*) Visualizing Data using t-SNE.

McInnes et al. (2018, *arXiv*) UMAP: Uniform Manifold Approximation and Projection.

Jawahar et al. (2019, *ACL*) What Does BERT Learn about the Structure of Language?

Resources

[How to Use t-SNE Effectively \(Distill\)](#) | [UMAP Documentation](#)

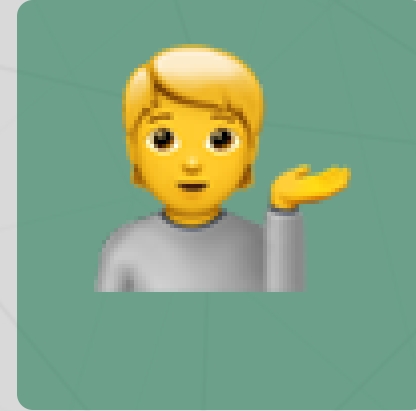
Questions?



Email me



Join our Discord



Come to office hours

Up next

Friday: Cognitive models of semantic representation!